YAPAY ZEKÂ ÇÖZÜMLERİ GELİŞTİRME KARİYER ATÖLYESİ

BTK AKADEMİ

GÜN 5

LangChain ile RAG Altyapısı Geliştirme

Belge → Chunk → Embedding → ChromaDB → Anlamsal Arama

07.11.2026 • 11.00–17.00 • 300 dakika net uygulama

Bugün ortaya çıkacak ürün

Bir sorudan ilgili kurumsal kaynağa giden çalışan, yerel arama sistemi.

SORU “Kulaklığımı kaybettim; hangi birime gideceğim?”

Kurgusal TXT — 9 belge → **Document** — metin + kaynak → **Chunk** — parçalara ayır → **Embedding** — yerel model → **ChromaDB** — kalıcı indeks

GÜN SONU Aranan kaynak: kayip_esya.txt • Sonuç: ilgili metin parçası + metadata

Müfredattaki 8 kazanımın tamamı

Her satır bir kod uygulamasına veya teknik karşılaştırmaya bağlanıyor.

01–02 Problem + belge hazırlığı

RAG'in neden gerekli olduğu; 9 TXT kaynak ve Document şeması.

03–04 Chunking + embedding

Overlap deneyleri, model seçimi ve yerel vektör üretimi.

05–06 Vektör DB kayıt + kıyas

Chroma indeksleme; Chroma–FAISS–pgvector karar kriterleri.

07–08 Arama + filtreleme

Similarity search, metadata filter ve retrieval kalite kontrolü.

300 dakikalık gerçek atölye akışı

4 × 75 dakika uygulama; 60 dakika mola.

01 · Belgeler

Preflight • TXT • Document • metadata

02 · Parçalar

Chunking • overlap • embedding

03 · İndeks

Vektör DB karşılaştırma • Chroma

04 · Arama

Similarity search • filtre • değerlendirme

KURAL Her blokta öğrenci kodu yazar; bir kontrol çıktısı görmeden sonraki aşamaya geçmeyiz.

RAG hangi problemi çözer?

Model, özel kurum yönergelerini eğitim verisinden bilmek zorunda değildir.

LLM’yi tek başına kullanırsak

Kurgusal kampüsün kayıp eşya prosedürünü bilmiyor olabilir.

Tahmin ederek yanlış bir danışmaya yönlendirebilir.

RAG ile arama yaparsak

Kurumsal belgeyi indeksleriz.

Soruya en ilgili parçayı ve hangi dosyadan geldiğini buluruz.

SINIR

RAG veri yolu ve sorgu yolu

İndeksleme bir kez; arama her soruda çalışır.

A İNDEKSLEME

TXT	Document	Chunk	Embedding	ChromaDB
Kaynak	metadata	420 / 80	embed_documents	diske kaydet

B SORGULAMA

Soru	Query vektörü	Benzerlik	Top-k sonuç	Kaynak
Türkçe metin	embed_query	vektör uzayı	ilk 4 parça	source alanı

Veri seti: 9 kurgusal kampüs dokümanı

Bütün öğrenciler aynı örnek dosyalarla aynı deneyleri yapacak.

`kayip_esya.txt`	Kayıp kulaklık, danışma, kimlik kartı
`kutuphane.txt`	Açılış saatleri, ödünç kitap
`wifi.txt`	Ağ erişimi ve destek masası
`sinavlar.txt`	Bütünleme ve sınav itirazı
`kulupler.txt`	Kulüp başvurusu ve etkinlik
`yemekhane.txt`	Öğünler ve menü
`ulasim.txt`	Kampüs ulaşımı
`staj.txt`	Staj süreçleri
`erasmus.txt`	Uluslararası değişim

ETİK Bu dosyalar gerçek üniversite düzenlemeleri değil; ders için üretilmiş sentetik örnekler.

Kod haritası: hangi dosya ne yapıyor?

Dosya adı → giriş → çıktı eşleşmesi; tümü öğrencinin ZIP paketinde.

`config.py`	model ve klasör ayarları	→	ortak değişkenler
`d5_01_documents.py`	TXT + meta.json	→	Document listesi
`d5_02_chunks.py`	Document listesi	→	chunk listesi
`d5_03_embeddings.py`	metin / soru	→	vektör
`d5_04_index.py`	chunk + embedding	→	Chroma indeksi
`d5_05_search.py`	soru + filtre	→	ilgili parçalar

Kurulum: komutlar ve anlamları

Terminal: projenin BTK_RAG klasöründeyiz.

WINDOWS POWERSHELL • ÖN HAZIRLIK

```
01  py -3.11 -m venv .venv
02  .\.venv\Scripts\Activate.ps1
03  python -m pip install -r requirements.txt
04  ollama pull embeddinggemma
05  ollama list
06  python 00_kurulum_kontrol.py
```

KODU OKUYALIM

venv

Bağımlılıkları sistem Python'undan ayırır.

requirements

LangChain + Chroma entegrasyonlarını kurar.

ollama pull

Embedding modelini yerel diske indirir.

ön kontrol

Paketleri, API'yi ve modelleri yoklar.

ÖNEMLİ Model indirmeleri sınıftan önce yapılmış olmalı. PowerShell engellerse .venv içindeki python.exe'yi kullan.

Preflight: çalışıyor mu, nasıl anlayacağız?

00_kurulum_kontrol.py başarı/eksik durumunu ayırır.

Çalışır durumdaki belirtiler

✓ langchain_core ve entegrasyonlar içe aktarılır
✓ Ollama /api/tags erişilebilir
✓ embeddinggemma listelenir
✓ qwen2.5:1.5b, 6. güne hazırdır

Hata varsa sırayla bak

1. Terminal doğru .venv içinde mi?
2. Ollama açık mı? /api/tags yanıtlıyor mu?
3. ollama list içinde model var mı?
4. requirements.txt kurulmuş mu?

ADRES http://localhost:11434/api/tags → varsayılan yerel Ollama servis kontrolü.

Neden iki farklı model var?

Embedding modeli ile sohbet modelinin görevini karıştırmayalım.

embeddinggemma

Girdi: metin veya soru
Çıktı: sabit uzunlukta sayısal vektör

5. gün aktif kullanacağız.

qwen2.5:1.5b

Girdi: mesajlar + bağlam
Çıktı: doğal dil cevabı

6. gün yanıt üretimi için kullanılacak.

YANILGI “Embedding = modele soru sormak” değildir; embedding bir arama temsili üretir.

LangChain Document nesnesi

Belge artık düz string değil; içerik ve kaynak bilgisini birlikte taşır.



GEREKÇE Chunk üretildiğinde metadata korunursa bulunan parçanın kaynağı kaybolmaz.

meta.json nereden geliyor?

Kaynak dosyanın hangi birime ait olduğunu dış yapılandırmada tutuyoruz.

meta.json • KURGUSAL ÖRNEK

```
{
  "kayip_esya.txt": {
    "birim": "Ogrenci_Isleri",
    "erisim": "genel"
  }
}
```

KODU OKUYALIM

Anahtar

Dosya adıyla birebir eşleşir.

birim

Metadata filtrelemede kullanılacak.

erisim

Ders için etiket; tek başına yetki kontrolü değildir.

TASARIM Document.metadata içine ayrıca source alanı ekleyeceğiz.

LAB 1 • TXT dosyalarını açma

d5_01_documents.py — hazır satırları incele, TODO'ya henüz dokunma.

d5_01_documents.py • HAZIR KOD

```
01  metadata = json.loads(
02      (ROOT / "meta.json").read_text(encoding="utf-8")
03  )
04  docs = []
05  for path in sorted(DATA_DIR.glob("*.txt")):
06      content = path.read_text(encoding="utf-8").strip()
07      if not content:
08          continue
```

KODU OKUYALIM

json.loads(...)

JSON metnini Python sözlüğüne çevirir.

DATA_DIR.glob

data/ içindeki TXT dosyalarını seçer.

read_text

Dosyayı UTF-8 okur; strip boşlukları temizler.

continue

Boş belgeyi indekslemeye sokmaz.

LAB 1 • TODO-1 ve TODO-2

Öğrenciler iki kritik satırı kendileri yazacak.

TODO YERİNE YAZILACAK KOD • KONTROL CEVABI

```
01  meta = {"source": path.name, **metadata[path.name]}
02  docs.append(
03      Document(page_content=content, metadata=meta)
04  )
```

KODU OKUYALIM

path.name

Örn. "kayip_esya.txt"; source alanı.

**metadata[...]

birim ve erisim alanlarını sözlüğe yayar.

Document(...)

Metin + metadata beraber saklanır.

ÖNCE DENE

LAB 1 • Çalıştır ve sonucu kanıtla

İlk kontrol: 9 belge ve her belgenin kaynağı geliyor mu?

1 DOSYAYI AÇ

d5_01_documents.py

2 ÇALIŞTIR

python d5_01_documents.py

3 KONTROL ET

“Belge sayısı: 9”
Dokuz dosya adı, birim bilgisi ve karakter sayısı görünür.

İLERİ GÖREV source alanını kaldırınca hangi özellikler bozulur?

Chunking: belgeyi neden bölüyoruz?

Arama birimi her zaman tüm dosya olmak zorunda değil.

TEK BÜYÜK BELGE

Kütüphane: açılış saatleri, ödünç, rezervasyon, VPN…

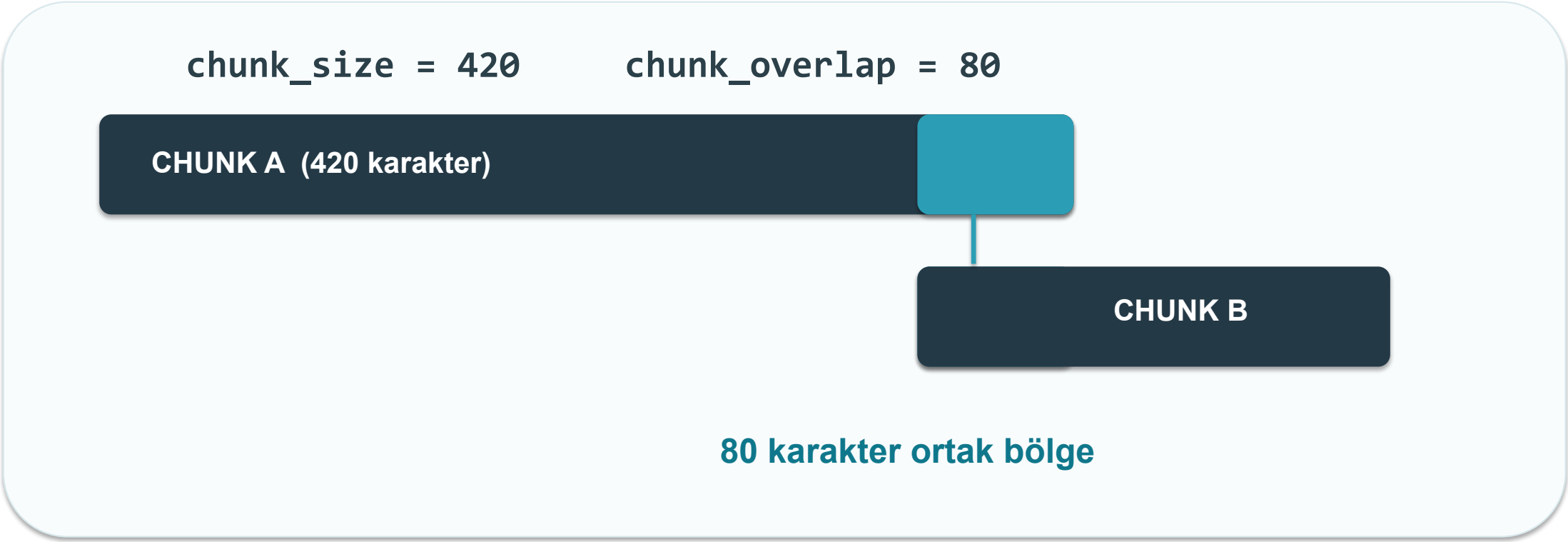
DAHA KÜÇÜK ARAMA PARÇALARI

chunk 0	Açılış saatleri
chunk 1	Ödünç ve uzatma
chunk 2	Rezervasyon / VPN

DENGE Çok küçük chunk bağlamı koparır; çok büyük chunk aramaya gereksiz bilgi taşır.

Overlap: iki parça neden kesişiyor?

Önemli bilgi iki chunk sınırında kopup kalmasın.



DENEY Overlap 0 olursa bazı parçalar sınırdaki anlamı kaybedebilir.

LAB 2 • Chunking kodunu yazalım

d5_02_chunks.py — TODO-3 / TODO-4.

d5_02_chunks.py • TODO ÇÖZÜMÜ

```
01  splitter = RecursiveCharacterTextSplitter(
02      chunk_size=chunk_size,
03      chunk_overlap=chunk_overlap
04  )
05  chunks = splitter.split_documents(docs)
```

KODU OKUYALIM

RecursiveCharacter...

Ayırma stratejisi; karakter temelli.

chunk_size

Hedeflenen en yüksek parça boyutu.

chunk_overlap

Komşu parçalar arası bağlam payı.

split_documents

Page_content bölünür, metadata taşınır.

LAB 2 • Aynı veriyle iki deney

Bir hiperparametre deneyi: çıktı sayısı ve sınırlar nasıl değişiyor?

Deney A

chunk_size=420
chunk_overlap=80

Daha ayrıntılı parçalar

Deney B

chunk_size=800
chunk_overlap=100

Daha uzun bağlam

ÖLÇ Çıkan chunk sayısını, bir chunk metnini ve source/chunk_id metadata alanlarını karşılaştır.

LAB 2 • Metadata parçalara geçti mi?

Arama sonucunu açıklayabilmek için chunk kimliği gerekir.

KONTROL KOMUTU VE ÇIKTI FORMATI

```
python d5_02_chunks.py

# Beklenen biçim (sayılar değişebilir):
Chunk sayısı: ...
kayip_esya.txt:000 -> 'KAYIP EŞYA...'
...
```

KODU OKUYALIM

source

Orijinal dosyanın adı.

chunk_id

Dosya + sıralı parça kimliği.

kontrol

len(chunks)>0 ve her chunk'ta source var.

İLERİ SORU chunk_id metadata'ya eklenmezse Chroma ID hangi başka şekilde izlenebilir?

Embedding: metnin vektör temsili

Anlam yakınlığı, aynı kelimeleri kullanma zorunluluğunu azaltabilir.

“Kulaklığım kayıp.” → `[0.12, -0.08, ...]`

“Kaybettiğim eşyayı arıyorum.” → `[0.11, -0.05, ...]`

“Bugün yemekte ne var?” → `[-0.32, 0.41, ...]`

NOT Sayılar kavramsal örnek; gerçek vektörü bugün embeddinggemma’dan alacağız.

Embedding nasıl iki tarafta kullanılır?

Belge vektörleriyle soru vektörünü aynı model uzayında karşılaştırırız.

İNDEKSLEME

```
embed_documents([...])
```

Birden fazla chunk → vektörler

ARAMA

```
embed_query("Soru")
```

Tek soru → tek vektör

Kritik teknik şart

Model/dim değişirse eski indeks uyumsuz kalır; yeniden oluşturmak gerekir.

Embedding modeli seçerken neye bakılır?

“En büyük model” değil, görev için dengeli model.

Türkçe performans

Parafraz ve yerel yazım çeşitliliği

Hız / bellek

Öğrenci bilgisayarında çalışabilme

Vektör boyutu

Depolama ve indeks maliyeti

Tekrar üretilebilirlik

Aynı sürüm, aynı benchmark soruları

SEÇİMİMİZ LAB için embeddinggemma: önceden indiriliyor; Türkçe soru setiyle sınıfta doğruluyoruz.

LAB 3 • İlk gerçek vektörü üret

d5_03_embeddings.py — TODO-5, model ve URL'yi config.py'den alır.

d5_03_embeddings.py • TODO ÇÖZÜMÜ

```python
from langchain_ollama import OllamaEmbeddings
from config import EMBED_MODEL, OLLAMA_URL

def embedding_client():
    return OllamaEmbeddings(
        model=EMBED_MODEL, base_url=OLLAMA_URL
    )
```

KODU OKUYALIM

OllamaEmbeddings

LangChain'in Ollama embedding sınıfı.

model

config.py → embeddinggemma

base_url

config.py → http://localhost:11434

return

Üretilen nesne başka lab'larda kullanılır.

LAB 3 • Model cevap vermek yerine vektör üretir

Beklenen: boyut bilgisi ve ilk birkaç sayısal değer.

ÖĞRENCİ TERMİNALİ

```
python d5_03_embeddings.py

# Çıktının biçimi:
Model: embeddinggemma
Boyut: <pozitif tam sayı>
İlk 5 değer: [0.0..., -0.0..., ...]
```

KODU OKUYALIM

Doğrulama

len(vector) > 0 ve sayısal elemanlar.

ConnectionError

Ollama servisi çalışıyor mu?

Model not found

ollama list / ollama pull embeddinggemma

YORUM Vektör değerleri kişiye/model sürümüne göre değişebilir.

Vektör veritabanı ne saklar?

Sadece vektör değil: belge içeriği ve izlenebilir metadata da gerekiyor.

ID	Vektör	Metin	Metadata
kayip_esya.txt:000	[0.12, −0.08, …]	Belgeden alınan parça	source, birim, chunk_id

KALICILIK persist_directory → Python kapansa bile indeks yerel diskte bulunabilir.

Güncel vektör veritabanları: güçlü seçenekler

2026 perspektifi · Ölçek, filtreleme, hibrit arama ve operasyon gereksinimine göre seçim

PLATFORM	MİMARİ	ÖNE ÇIKAN GÜÇ	HANGİ SENARYODA?
Pinecone	Yönetilen / serverless	Düşük operasyon yükü; yönetilen vektör araması	SaaS tercih eden hızlı ürün ekipleri
Milvus / Zilliz	Açık kaynak / Cloud	Dağıtık mimari; çok büyük vektör koleksiyonları	Yüksek ölçek, kümeli altyapı
Qdrant	Açık kaynak / Cloud	Payload indeksleri; zengin metadata filtreleri	Self-host ve ayrıntılı filtre ihtiyacı
Weaviate	Açık kaynak / Cloud	BM25 + vektör hibrit arama	Hem sözcük hem anlam araması
Elasticsearch	Arama platformu / Cloud	Tam metin + kNN + hibrit sıralama	Kurumsal arama ekosistemi
PostgreSQL / pgvector	PostgreSQL eklentisi	SQL + HNSW / IVFFlat; ilişkisel veriyle bütünleşme	PostgreSQL altyapısı mevcutsa

DERSTE **Chroma ile uygulama. FAISS bir vektör arama kütüphanesidir; tam bir DB değildir.**

"En güçlü" seçim veri hacmine, arama türüne, güvenlik ve işletim bütçesine bağlıdır.

LAB 4 • Belge ve ID'leri hazırla

d5_04_index.py — önceki üç LAB'in çıktıları burada birleşiyor.

d5_04_index.py • HAZIR SATIRLAR

```
01  docs = make_chunks(load_documents())
02  ids = [d.metadata["chunk_id"] for d in docs]
03
04  # Örnek ID:
05  # "kayip_esya.txt:000"
```

KODU OKUYALIM

load_documents

TXT + metadata yükler.

make_chunks

Metni böler, chunk_id ekler.

ids

Her parçaya izlenebilir ve tekil kayıt kimliği verir.

KONTROL len(ids) == len(docs) olmalı; tekrar eden ID olmamalı.

LAB 4 • Chroma.from_documents parametreleri

d5_04_index.py • TODO-6 ÇÖZÜMÜ

```
store = Chroma.from_documents(
    documents=docs,
    ids=ids,
    embedding=embedding_client(),
    collection_name=COLLECTION,
    persist_directory=str(DB_DIR),
)
```

KODU OKUYALIM

documents / ids

Kaydedilecek chunk'lar / tekil anahtarlar.

embedding

Vektörleri üreten embedding istemcisi.

collection_name

Aynı indeks için mantıksal koleksiyon adı.

persist_directory

Kalıcı Chroma dizini: chroma_kampus/

LAB 4 • İndeksi kur ve doğrula

Kalıcı dizinin ve koleksiyonun gerçekten oluştuğunu gör.

1 DOSYAYI AÇ

d5_04_index.py

TODO-6 tamam. İlk kez embedding + Chroma birlikte çalışır.

2 ÇALIŞTIR

python d5_04_index.py

3 KONTROL ET

“Kaydedilen chunk: ...”
“Chroma dizini: .../chroma_kampus”
“Koleksiyondaki kayıt: ...”

İLERİ GÖREV Klasördeki veritabanını terminalden yeniden açınca sonuç korunuyor mu?

İndeksleme: hangi hatayı nasıl ayırırız?

Üç farklı katmanı kontrol etmek, rastgele yeniden kurmaktan iyidir.

Model yok	Ollama 404 / model bulunamadı	ollama list / ollama pull embeddinggemma
Koleksiyon sorunu	Eski / uyumsuz indeks	yalnız eğitim chroma_kampus dizinini yeniden kur
Eşsiz olmayan ID	duplicate ID / kayıt çakışması	chunk_id üretimini incele

DİKKAT Veritabanı reset'i yalnız eğitim klasöründe; üretim verisine uygulanmaz.

Similarity search: sorudan belgeye

Chroma, soru vektörüne en yakın k parça döndürür.

Türkçe soru
Yeni ifade

→

embed_query
soru vektörü

→

Chroma arama
yakınlık hesapla

→

Top k = 4
4 aday

→

Document
metin + metadata

“Kulaklığım kayıp.” → kayip_esya.txt (beklenen ilgili kaynak)

SINIR Benzerlik sırası model ve veri ayarlarına bağlıdır; sonucu test edip ölçüyoruz.

LAB 5 • Var olan Chroma indeksini aç

d5_05_search.py — yeniden embedding üretmek değil, aynı koleksiyona bağlanmak.

d5_05_search.py • get_store()

```
01  return Chroma(
02      collection_name=COLLECTION,
03      embedding_function=embedding_client(),
04      persist_directory=str(DB_DIR)
05  )
```

KODU OKUYALIM

collection_name

İndekslemede kullanılanla aynı olmalı.

embedding_function

Soruya embedding üretmek için gerekir.

persist_directory

LAB4'te oluşturulmuş aynı dizin açılır.

ÖN KOŞUL chroma_kampus klasörü yoksa önce python d5_04_index.py çalıştırılır.

LAB 5 • Similarity_search

Sorguyu, adet sınırını ve opsiyonel filtreyi birleştir.

d5_05_search.py • TODO-7 ÇÖZÜMÜ

```python
def search(question, k=4, unit=None):
    filter_value = {"birim": unit} if unit else None
    return get_store().similarity_search(
        query=question, k=k, filter=filter_value
    )
```

KODU OKUYALIM

question

Sorunun doğal dil metni.

k=4

En fazla 4 ilgili parça istenir.

filter_value

unit verilirse metadata koşulu.

geri dönüş

Document listesi; her item kaynak taşır.

ÖĞRENCİ GÖREVİ İlk önce filtersiz aramayı çalıştır; sonra unit parametresiyle farkı gözlemle.

LAB 5 • Dört Türkçe soru ile test

Beklenen doküman kimliği bir oracle’dır; sıralamayı model belirler.

“Kulaklığımı nereden teslim alırım?”	kayip_esya.txt
“Cumartesi kütüphane kaçta açılır?”	kutuphane.txt
“Wi-Fi şifremi nasıl güncellerim?”	wifi.txt
“Sınav sonucuna nasıl itiraz edilir?”	sinavlar.txt

KONTROL python d5_05_search.py ile ilk sorguyu çalıştır; diğer soruları question değişkeninden dene.

Metadata filtering: aramayı daraltır

Semantik yakınlık ve yapısal filtre iki farklı koşuldur.

Soru: “Kaybolan kulaklığıma ne oldu?”

FİLTRESİZ

Tüm birimlerden benzerlik arar.

FİLTRELİ

Yalnız Ogrenci_Isleri kaynakları.

FİLTRE

```
filter={"birim": "Ogrenci_Isleri"}
```

LAB 5 • Filtreli / filtresiz karşılaştır

Aynı soru, iki farklı aday havuzu.

DENEY KODU • d5_05_search.py SONRASI

```
soru = "Kaybolan kulaklığımı nerede bulabilirim?"
print([d.metadata["source"] for d in search(soru)])
print([d.metadata["source"] for d in search(
    soru, unit="Ogrenci_Isleri"
)])
```

KODU OKUYALIM

Deney

Aynı soruyu iki kez gönder.

Gözlem

Filtreli sonuçlarda birim alanı eşleşmeli.

Ölçüm

Filtrenin sonuç sayısını ve çeşitliliğini kıyasla.

ÖNEMLİ Filtreli sorgunun döndürdüğü her Document için metadata["birim"] == "Ogrenci_Isleri".

Retrieval kalitesi nasıl ölçülür?

En az bir beklenen kaynak + sabit soru seti ile değerlendir.

Hit@k = beklenen kaynak ilk k sonuç içinde mi?

Hit@1

Beklenen kaynak ilk sıradaysa 1, değilse 0.

Hit@3

Beklenen kaynak ilk 3 sonuçtan birindeyse 1.

ÖRNEK İlk üç sonuç: wifi, kayip_esya, sinavlar → kayıp eşya için Hit@1=0; Hit@3=1.

İleri seviye görev: kontrollü ablation

Yalnızca bir parametreyi değiştir, aynı 4 soruyla değerlendir.

Deney tasarımı

A: chunk_size=420 / overlap=80
B: chunk_size=800 / overlap=100

Her senaryoda indeksi yeniden oluştur.

Raporlanacak sonuç

Chunk sayısı
4 soru için Hit@1 ve Hit@3
Süre farkı / kaynak sıralaması

Hangi ayar neden daha iyi?

METODOLOJİ Veri, sorgular ve embedding modeli sabit kalsın. Sayıları ölçmeden başarı iddia etmeyin.

Metadata filtresi ≠ veri yetkilendirmesi

Kurumsal kullanımda sınır, aramadan önce zorlanır.

Filtre ne yapar?

Arama sonuç kümesini yalnız belirli birim belgeleriyle sınırlar.

Örn. birim = Ogrenci_Isleri

Yetkilendirme ne yapar?

Kullanıcının görmeye yetkili olduğu belgeleri sunucu tarafında denetler.

Kullanıcıdan gelen unit parametresine tek başına güvenilmez.

ÜRETİM NOTU Gerçek kurumda satır/belge erişimi doğrulaması ve yetkisiz belge indeksinden kaçınma gerekir.

Sorun çıkarsa: 4 adımlık teşhis

Ders süresini kaybetmemek için tanıyı katman katman yap.

1	**Belge okunuyor mu?**	`python d5_01_documents.py`
2	**Chunk oluşuyor mu?**	`python d5_02_chunks.py`
3	**Ollama vektör üretiyor mu?**	`python d5_03_embeddings.py`
4	**İndeks ve arama çalışıyor mu?**	`python d5_04_index.py → d5_05_search.py`

YEDEK PLAN offline_yedek.py yalnız anahtar kelime aramasıdır; embedding/vektör DB/RAG yerine geçmez.

Gün 5 — baştan sona çalıştırma sırası

Aşağıdaki komutların tamamını öğrenci kendi makinesinde çalıştırmalı.

VS CODE TERMİNALİ • BTK_RAG PROJE KLASÖRÜ

```
python 00_kurulum_kontrol.py
python d5_01_documents.py
python d5_02_chunks.py
python d5_03_embeddings.py
python d5_04_index.py
python d5_05_search.py
```

Başarı ölçütü

9 belge
>0 chunk
>0 vektör boyutu
Chroma dizini
Kaynaklı top-k sonuç

Çıkış kontrolü: öğrencinin teslimi

Ders bittiğinde her öğrenci kodu ve kanıtını göstermeli.

01	9 TXT dosyası Document olarak okunuyor.
02	Chunking ve overlap ayarı değiştirilebiliyor.
03	Embedding modeli gerçek sayısal vektör döndürüyor.
04	Chroma kaydı disk üzerinde bulunuyor.
05	Similarity search ve metadata filter sonuç veriyor.

6. GÜNE DEVİR chroma_kampus klasörünü silmeyin; yarın Retrieve → Rerank → Generate bunun üzerinden ilerleyecek.

Kaynaklar ve teknik sınırlar

Sunumdaki sınıflar mevcut öğrenci ZIP paketindeki uygulamaya göre seçildi.

LangChain • Chroma	https://docs.langchain.com/oss/python/integrations/vectorstores/chroma
LangChain • Embeddings	https://docs.langchain.com/oss/python/integrations/embeddings
LangChain • Text splitters	https://docs.langchain.com/oss/python/integrations/splitters
Ollama API	https://docs.ollama.com/api
LangChain API Reference	https://reference.langchain.com/python/langchain-chroma/vectorstores/Chroma

ŞEFFAFLIK Slayt kodları referans çözümle eşleşir; gerçek Ollama–Chroma entegrasyonu ders makinesinde test edilmelidir.